

Database is a collection of ^{data} format that can be easily accessed.

→ DBMS is a software application used to manage our database. Data is an organized collection of data.

Types of databases

Relational

→ Data stored in tables
ex. MySQL

Non relational Databases

→ data not stored in tables.
ex. mongoDB

SQL (Structured Query language)

→ is a ^{Query} programming language used to interact with relational database.

is a standard query language that is used to communicate with relational databases and perform operations like create, update, Read, delete

→ It is used to perform CRUD operations.

Table is the combination of rows and columns where data is stored in them.

→ Columns → shows the structure/schema of table

→ rows → shows/contains the actual data
→ individual data.

DBMS is a software used to interact with database so that users can create, store, delete, update, manage data inside database.

Database is the organized collection of data that is stored electronically so that it can be accessed easily.

Primary key is unique identifier that identifies each row uniquely.
↳ is a column that uniquely identifies each row in a table.

Rules of primary key → must be unique
→ cannot be NULL
→ only one primary key for a table

→ Primary key uniquely identifies every row

Ex:

Student_ID	Name
101	Harish
102	Rahul
102	Rohit

Student_ID is primary key

Foreign Key: A foreign key connects two tables

↳ It establishes a relationship b/w tables

Ex: Customer Table

Cust-ID	Cust-Name
101	Harish
102	Rahul

Orders Table

Order-ID	Cust-ID (foreign key)	Product
1	101	Laptop
2	102	mouse

Cust-ID in orders table is foreign key.

~~NULL~~
NULL: means no value or value not provided

↳ does not have to be unique
→ must have a value

Unique:

- multiple unique columns can exist in a table
- A unique column cannot have duplicate values
- UNIQUE constraint ensures that no two rows contain the same value in that column.

NOT NULL: It must have a value
Does not have to be unique.

Comparison:

Primary Key

- A value
- unique value

PK = NOT NULL + UNIQUE

Ex. Aadhaar

NOT NULL

- A value must exist
- Duplicates are allowed

DEFAULT Constraint:

default constraint automatically inserts a predefined value when no value is provided for that column.

default VS NOT NULL

means,

if no value is given,

I will use this value.

No 'error'

means a value must exist

→ if no value is given, mysql gives error

Ex: Create Table Employee (Emp-name varchar(30),
Emp-Age tinyint,
Emp-country varchar(30) default
'India')

Check Constraint
Check constraint ensures that values entered into a column satisfy a specific condition.

Ex: Create table Students

```
(
  Stu-ID INT Primary Key,
  Name VARCHAR(50) NOT NULL,
  Age TINYINT CHECK (Age >= 16 AND Age <= 100);
```

ForeignKey: Is a column that refers to primary key of another table, creating a relationship between the two tables.

Ex: Customer Table

cust-ID	Name
101	Hanish
102	Rahul

Here cust-ID is primary key

order Table:

order-ID	cust-ID	Product
1	101	Laptop
2	102	Phone

Here i) order-ID is primary key

ii) customer ID is Foreign key

Because it points to cust-ID in customers table.

Ex: Create table orders

```
(
  orderID INT Primary Key,
  cust-ID INT
```

Product VARCHAR(50),

Foreign key (CustomerID)

References customers (CustomerID));

* Foreignkey can have duplicate values.

Ex:

order-ID	cust-ID
1	101
2	101

Data Redundancy:

for example we have order table & customer table

cust: ID	Name
101	Harish
102	Rahul

cust ID primary key

order-ID	cust-ID	Product
1	101	Laptop
2	102	phone
3	101	mouse

cust-ID foreign key

Here the customer with ID 101 has placed the order twice and order table does not store customer name.

If ~~customer~~ order table store the customer-name,

Suppose if the customer
change his name
from Harish to Harish
Kumar

order-ID	custName	Product
1	Harish	Laptop
2	Rahul	phone
3	Harish	mouse

then he need to be updated

in order 1, order 3, future orders, Invoice etc.

So Instead we store only cust-ID=101,

if he change his name we can change it in only one row in custo-table.

customer-ID	Name
101	Harish Kumar

every order still points to cust-ID 101, nothing changes

→ This is called Data Redundancy

Redundancy means storing the same information again and again.

Ex:

Order-ID	cust-Name
1	Harish
2	Harish
3	Harish

name 'harish' is repeated many times, that is redundancy

for example, marks table

Subject	Student Name
DBMS	Harish
SQL	Harish
Python	Harish

con-

Subject	ID
DBMS	101
SQL	101
python	101

So from both Student-ID is best.

→ If student name changes, we update it once in student table

→ So this idea has a name called "NORMALIZATION"

NORMALIZATION: Reduces the unnecessary duplication of data.

Referential Integrity:

⇒ eppudu mana dhajara oka table customer table undhi

cust-id	cust-name
101	Harish
102	Rahul

eppudu, orders table

order-id	cust-id	Product
1	101	
2	102	
3	101	

-eppudu okavul customer table lo Harish aney oka customer ni delete chesane amuko, eppudu em aythadi, ~~harish~~ aney customer delete aythadu.

Kani order table lo unna cust-id 101, 101 records ~~aney~~ aneyi undu

So dhini vatta problems varthayi; so to overcome "the database rejects the delete to protect relationship"

Example Bank

cust-ID	Name
101	Harish

loans

loan-ID	cust-ID
5001	101

eggeudu okavela ~~bank~~ cust-ID 101 ni delete chestundha ante no endukante tuka loan anedhi pending or kattali kabati kabati dhini kosame oka concept undhi adhe,

"Referential Integrity"

→ means that relationships between tables must be always remains valid.

Relationship Types:

1) one to one

Ex: person ↔ passport

one passport belongs to one person.

2) one to many

customer → many orders

3) many to many:

Student ↔ courses

one student can have many courses and ~~many~~ one course have many students.

Ex: Harish → SQL
Python
EXCEL

SQL → Rahul
Harish

create database companyDB; → It selects the database, & the commands which are executed next after this, are executed/creates tables in companyDB.

SQL server executes commands in currently selected database.

To create a table:

create table table-name (Col name datatype,
Col name2 datatype,
Col name3 (datatype constraint))

Ex. Create table Employee

(Emp_ID INT PRIMARY KEY,
EM_Name Varchar(25) NOT NULL,
mail Varchar(100) UNIQUE NOT NULL,
Age INT NOT NULL CHECK (Age > 18),
Salary INT NOT NULL CHECK (Salary > 70),
DOB DATE NOT NULL,
Country Varchar(20) DEFAULT 'INDIA');

Insert: if inserts the values into tables

~~The syntax~~ Insert into table, table name (Colname, Colname/
values(., ., .))

Ex: Insert into table Employee (EmpID, EmpName, mail, Age, Salary, D.O.B, Gender)

Values (10, 'Harish', 'hari@gmail.com', 22,
15,000, 2021-01-01, 'Indra')

meth-d 2)

Insert into Employee values (102, 'Rahul', 'Rah@gmail.com',
24, 70000, '2021-02-02')

for "default" entry:

• Insert into Employee (emp-id, emp-name, mail, Age, Salary,
DOB)

Values (103, 'Hari', 'Ha@gmail.com', 25, 15000, '2020-04-03')

Here it enters country column default by "INDIA".

(or)

Insert into ... (... , country)

Values (104, 'Ra', 'R@gmail.com', 24, 15000, '2021-04-02',

DEFAULT)

⇒ Here if we mention country column in Insert, in value we keep it default so that SQL server enters default value.

SELECT

Select statement is used to retrieve (read) data from one or more tables.

→ It only reads the data. Syntax: Select * from tablename;

Ex: Select * from Employee.

"Select" ⇒ means I want to retrieve some data

"*" ⇒ All columns.

"From" ⇒ read data from

Employee ⇒ table name.

So, basically it means select all columns from Employee table

"Result set" ⇒ is the data returned by a SELECT query.

per suppose: we have table Employee.

Emp-ID	Name	Salary
101	Janku	18,000

So to see only employee name we use

Select * from emp } that displays only name

When we execute

Select EmployeeName
From Employees;

SQL Server thinks like
User wants EmployeeName



Open Employees table



Ignore other columns



Read EmployeeName



Display Result set



Done

DDL, Create
alter
rename
truncate
drop

DML - Update, delete, Insert

Why should we avoid select *
in large databases

Ans) Selecting only required
columns makes queries
clearer and easier to
maintain.

→ no modification of table.

WHERE Clause (Filtering Data)

Syntax: Select * from table-name
where condition;

It is used to show the specific records for example
records from employee table where employee name is Harish.

Ex: Select *
from Employees
where EmployeeName = 'Harish';

word-by-word

Select - I want to
retrieve data

*

→ Show all Columns

from employees
read from emp table

where → only if this
condition is true

Internally, the server checks the
employee table where emp-name = Harish
and display it.

- checks 1st row, 2nd row, 3rd
display which is apt.

Ex: Select * From Employees
where Salary > 30,000 } It displays all records in
the employee table whose salary
is greater than 30,000.

Comparison Operators

operator	meaning	ex.
=	equal to	Salary = 50,000
>	greater than	
<	less than	
>=	greater than equal to	
<=	less than or "	
<> or !=	not equal to	Country <> 'USA'

"AND" Operator

AND means both condition must be true

Ex: like True & True → True

True AND False → False

F AND True → False

F AND F → False

only True AND True → True

Ex: Select * from Employee

where Salary > 40,000 and Country = 'India';

"OR" operator:

Any one condition can be true.

Truth-table

True OR True True

True OR False True

False OR True True

False OR False False

→ only false false is rejected

Ex: Select * from employees
where country = 'India'
OR salary > 45000;

It checks from table employees that country = 'India' in first row, then second, 3rd, ...
~~if any row~~
checks each row if any one condition is true it displays the result.

"NOT" operator

NOT means opposite (opposite of condition)

Select *
From Employees
where NOT country = 'India';

SQL Executes in the

order of FROM



WHERE



SELECT



① open from Employee table

② where salary > 45000 filter rows

NOT like Select
X from
where

③ Step ③ → select employee name
Display only requested

ORDER BY

order by is used to sort the result set in ascending or descending order.

⇒ Suppose if we want to see employees from high salary to lowest salary.

~~(order by only sort the result set)~~

order by doesnot sort the table permanently.

It only sorts the output you see.

employee
first

Ex: Harish 95

Rahul 70

Sai 88

Priya 60

If the principle says arrange from highest to lowest
Teacher does not change students,
only changes order of list.

Ex: Select *

From Employees

order by Salary;

Select
retrieve data

From employees

Open employee table

~~SQL~~ order by salary

Arrange the result according to salary

By default

Ascending order is followed by SQL server

like 10, 20, 30, 40, 50. --

numbers: Small $\rightarrow$ large.

names: A B C D alphabetical order.

Dates: old to new.

Select * from employees } output
order by salary

40,000
45K
50K
55K

ASC Keyword Select *

from employees

order by salary ASC;

DESC (Descending)

Select * from

employee

order by Salary DESC;

output: 55K

50K

45K

40K

-- largest $\rightarrow$ smallest

ex. Query:

Select employee name, salary
From employees
where salary ≥ 45000
order by salary Desc;

Table ID	Name	Salary	Country
1	Harish	50,000	IND
2	Rahul	45000	IND
3	Sai	55000	USA
4	Priya	40000	IND

SQL server executes the query in order

From employees

i) $\rightarrow$ it opens employees table

ii) Checks condition

where salary ≥ 45000

for each row and removes priya, stores remaining data in temporary table

like

Harish	50,000
Rahul	45000
Sai	55000

iii) order by Salary DESC

$\rightarrow$ it does not go to original table

it sorts only in temp-table

like

Sai	55000
Harish	50000
Rahul	45000

iv) select

It displays the only selected columns from temporary table.

Top: where filters data.

order by sorts the filtered data

"TOP" clause

→ Suppose a Data Analyst want to see top-5 highest paid employees from Amazon where there are a lakh employees

for these we do not use "where" x

" , " , "order By" x

order by can sort the employees, but doesn't limit how many rows are returned.

To limit the numbers of rows, we use ~~SQL~~ "TOP"

what is "TOP"?

TOP is used to limit the number of rows returned by a query.

Ex: Select TOP 3 * From Employees; } this example

Shows first 3 rows from employee

TOP 3 only directly shows only which employees are top but not with highest salary.

TOP + ORDER BY

Manager says: Show me top 3 highest paid employees

we need two things

1) Sort by salary

2) Take only first 3 rows

Ex: Select TOP 3 *
From Employees
ORDER BY SALARY DESC;

SQL Server executes it like

- 1) Open employees table
- 2) no where clause, keep all rows
- 3) sort by salary (high to lowest)
- 4) Apply TOP 3
- 5) display result

* Top without order by is unpredictable

* Top with order by is meaningful results

Ex. Select Top 5 * From Employees
where emp-sal > 35000 order by
Salary desc;

→ This Query executed
in the following way

i) from employees table

ii) applying where condition

iii) Sorting the Salary

iv) select the Top 5

v) display the top 5 salary

* without "order by" we cannot do this
like arranging Top 5 highest paid employees

⇒ By using where we can only filter
the data, but the order is not
right so get a perfect order

either Asc or Desc we use order by.

Like example. Virat Kohli Scores 100, 80, 85, 98, 90, 20.

If we want top 2 highest Scores,
when we apply only where on score like score > 40, then
we have 100, 80, 85, 90. But not in an order.
So when we apply sort we can get order 100, 90, 85, 80 &
then Top 2.

DISTINCT

Distinct is used to return only unique values by removing duplicate rows from result set

⇒ It does not delete data from table

⇒ It only remove duplicates from result set

Ex: Select distinct city
From customers;

Select distinct country, Age
from employees

Select distinct blood group
from hospital

where → filter rows

order by → sort rows

Top → limit rows

Distinct → removes duplicates
values from result set

Ex: we have Table

EMP_ID	EMP_NAME
1	Rahul
2	Rahul
3	Rahul

Query: Select distinct empID, emp-name from emp

⇒ SQL server does not check only one column, it check the entire row like

Row 1	1, Rahul	} not identical, emp-ID is differe
2	2, Rahul	
3	3, Rahul	

∴ It shows →

* [↑] **Rule** Distinct removes duplicate rows based on all columns in select list, not just one column.

Aggregate Functions:

COUNT() → Count no. of records

SUM() → Add values

AVG() → Find Average

MIN() → Smallest value

MAX() → largest value

If a table has 100 ~~columns~~ employees, to count total no. of employees we use:

Select Count (*)

From employees;

} It counts all the rows from employees and return the result

Select Count(emp-name)

from employees

} It counts the no. of employees from employee table & shows it

Select Count(*) as total

from employees

} It counts all the ~~rows~~ columns and shows it as 'total' column name

SUM()

Suppose you are working in company & manager ask that how much money we are paying every month for employees.

→ So we use select (*) x

→ Count() x → It tells only no. of employees.

for that purpose we use SUM()

Definition: sum() is an aggregate function that adds all numeric values in a column and returns total.

Ex: Table

Emp

Har

Roh

Uda :

Salary

50K

45K

55K

SUM Query: select SUM(salary)

from employees;

SQL Server executes it in from table employee

read salary column

add all salary

return one number (sum)

SUM with WHERE

→ Suppose if you manager asks what is total salary paid to only employees in India?

for that: EX: SELECT SUM(salary)

from Employees

where Country = 'India';

SQL Server: From employees

↓
where Country = 'India'

↓
Keep India emp → Add salaries → Return total

SUM only works with numeric columns such as 1) INT

EX: Total orders

select sum(orders.Amount)

from orders;

Bank Balance: select sum(Balance)

from account;

AVG:

Suppose your manager asks what is avg salary of our employees

then we don't calculate manually.

→ we have function called "AVG".

ex: Select AVG(Salary)
from Employees;

AVG is calculated using $\frac{\text{SUM}(\text{Salary})}{\text{Count}(\text{Salary})}$
= Average.

SQL Server executes:

from table employees

↓

reads salary column

↓

total salary

↓

total emp → AVG(Sal) → return result (return one value)

AVG with WHERE

"What is average salary of employees from India?"

Query: Select AVG(Salary) from employees where Country = 'India';

Difference b/w Count, SUM, AVG

Count no. of rows

SUM(Sal) Total Salary

AVG(Sal) Average Salary

MIN() and MAX()

→ Suppose your manager asks who has highest salary, lowest salary

→ So you don't search manually thousands of records.

→ for that SQL has two functions

MIN() → finds smallest value

MAX() → finds largest value

Ex Table:- Employee

Employee	Salary
Hari	50K
Rahul	45000
Sai	55K
Priya	40K
Anil	70K

ex: Query `Select max(salary)`
`From Employees;`

Output: 50,000

`Select min(salary)`
`from employees`

Op: 40000

These functions returns max, min value of entire ^{selected} column, not entire row.

→ min and max can be applied on varchar columns.

Why because, strings can be arranged alphabetically.

Suppose Ex. Name

Harsh

if we execute

we have

Rahul

`Select max(employee name)`

Sai

Anil

from employee

as largest alphabetically

Sai

" min → Anil " smallest "

Comparison

function

numeric

varchar

Count()

✓

✓

Sum()

✓

✗

Avg()

✓

✗

min()

✓

✓

max()

✓

✓

Why does this work?

Because SQL Server knows how to compare strings alphabetically

ex: $A < B < C < D < E < \dots < Z$ so, min() → first alphabetically
 max() → last "

→ So instead of total salary of all employees, we can

if managers want total salary as dept-wise or countrywise.

Group By:

Suppose we have a table

emp	dept	Salary
Hari	IT	50K
Roh	IT	45K
Jai	HR	55K
Priya	HR	40K

If manager asks "what is total salary of IT employees?"

for this we can use where: `select sum(salary) from employees`

where dept = 'IT';

⇒ Now if manager asks "show me total salary for each dept."

for each dept it will be O/P

Dept	Total Salary
IT	95000
HR	95000
Sales	70000

⇒ If we use where, it takes

3 queries that too different

queries.

⇒ So to overcome this problem SQL has "GROUP BY"

EX: `select Dept, sum(salary) from employees group by Dept;`

O/P

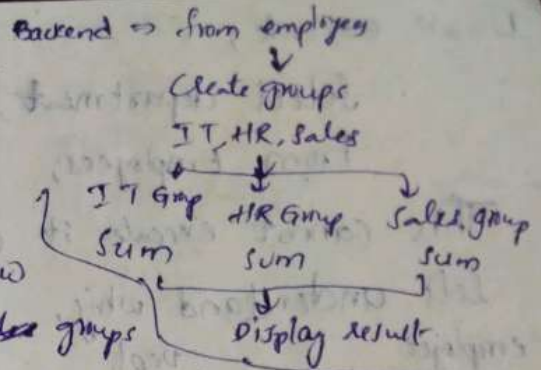
Dept	Total Salary
IT	95K
HR	95K
Sales	70K

where gives only specific or which we needs or condition is reqd
whereas "GROUP BY" helps to group the same data or
how do I want divide the data.

EX: Group By Dept } → makes one group for IT, HR, Sales.

Group by is used to group rows that have the same values in specified column, so that aggregate functions can be applied to each group separately.

Ex: Select Dept, Sum(Salary)
from Employees
Group by Dept;



SQL Server does not create new tables, it creates temporary ~~table~~ groups in memory, performs calculations, return results then these groups are disappears from memory.

"Temp Difference"

Distinct + Group By

only gives unique values, it eliminates duplicate values

↳ makes groups of similar data

Ex: Some IPL Players

if I ~~use~~ ^{apply} distinct on ~~players~~ ^{teams} it gives RCB, MI, CSK.

Players	Team
Virat	RCB
Rajat	RCB
Rohit	MI
Surya	MI
Dhoni	CSK

If I ask how many players in each team? group By

Team	Players
RCB	2
MI	2
CSK	1

Total Sal
95K
95K
70K

is right.
or

SQL TOOLBOX

filter rows - where

removes duplicate values - Distinct

makes groups for calculations - Group by

Sort data -> order by

Result only -> Top first N rows

Count rows -> Count()

Total values -> Sum()

Average -> Avg()

Smallest value -> Min()

Largest value -> Max()

If we execute;

```
select department, sum(salary)
from employees;
```

This cannot execute it why because
lets understand why,

employee	dept	salary
Harish	IT	50K
Rahul	IT	45K
Sai	HR	55K
Priya	HR	40K
Anil	Sales	30K

⇒ now look at Query.

Step-① reads the tables

② Calculates the sum

but now it ask which department should i display the result.

dept - ~~IT~~ sum 26K X wrong

dept HR sum 26K X wrong

dept Sales sum 26K X wrong

dept IT, HR, sales sum 26K X wrong

⇒ SQL Server Confuses

It says you asked me to display dept, but you didn't tell
how to group department.

So it throws an error:

⇒ to solve this

we tell SQL Server: group the rows by dept first

Query: select dept, sum(salary)

from employees

Group by department;

It creates groups: IT group → 95K

HR group → 95K

Sales group → 70K

The output becomes:

Dept	SUM(salary)
IT	95K
HR	95K
Sales	70K

Golden Rule

Whenever you select a normal column together with an aggregate function, the normal column must usually appear in the GROUP BY clause.

Suppose you are data analyst

Your manager ask "Tell how many emp working in each dept."

Table:

Employee	Dept
Hanish	IT
Rohith	IT
Rahul	IT
Sai	HR
Priya	HR
Kiran	Sales

Can use where → NO

distinct → NO

count(*) → NO

Select dept, count(*)

from employees

Group by Department;

In English:

Group employees by Department,

then count employees in each dept.

Step 1 → open Emp table

2) Create groups

IT	HR
HR	S
	P

Sales → Kiran

Step 4 Display result

Dept	Count
IT	3
HR	2
Sales	1

3) Apply count(*) to each group

IT - 3

HR - 2

Sales - 1

Important rule: when you use
 Count(), min(), max(), avg(), sum()
 together with a normal column,
 that normal column must be in group by clause

Ex: Select dept, Count(*)
 from employees
 Group by Dept;
 ✓ correct

Select dept, Count(*)
 from employees;
 ✗ wrong

Ex: Select Country, Count(*)
 from customers
 Group by Country;

Select category, sum(Quantity)
 from products
 Group by Category;

Any column in 'select' clause that is not inside
 an aggregate function must also be included
 in group by clause.

HAVING clause

Suppose we have students marks table,

If manager asks show only students whose marks are greater than
 then we use where condition (where marks > 80)

If manager asks, show only those courses whose average marks > 80
 Here he is not asking individual students,
 he is asking about groups.

Calculate for ex:

SQL - 83

Python - 80

Excel - 81

now show only courses avg > 80

O/p must be like this

Course	Avg
SQL	83
Python	80
Excel	81

So can we write where $\text{avg}(\text{marks}) > 80$, no it will not execute

AS SQL server executes like $\downarrow$

from
 $\downarrow$
where
 $\downarrow$
group by
 $\downarrow$
Aggregate function
 $\downarrow$
Select

Select course, Avg(marks)
from students
where Avg(marks) > 80
Group By course

Here, WHERE executes before GROUP BY
and AVG(marks) is calculated after GROUP BY

So SQL server gives an error, it first checks and
 $\text{avg}(\text{marks}) > 80$, then it sees
no calculated, so it shows error
 $\text{avg}(\text{marks})$ not calculated

So to overcome this we use "HAVING" (each word)
mainly appears then use)

Ex. Select course, AVG(marks)
from students
Group By course

Having $\text{AVG}(\text{marks}) > 80$;

now "where" became "having"

Difference between where
filters rows

executes before group by

cannot use $\text{AVG}()$, $\text{SUM}()$, $\text{DISTINCT}()$
directly

4 having

filters groups

executes after group by

can use Aggregate functions

Where $\rightarrow$ individual rows

Having $\rightarrow$ Entire Groups

Remember

in easy way: A school of 100 students

if principal asks show students > 80 , then we use where

if principal asks show classes whose average are > 80 .
You must check $\text{CLASS A}(\text{AVG})$, $\text{CLASS B}(\text{AVG})$ then compare

SQL Server Execution Order

FROM
↓
WHERE
↓
GROUP BY
↓
AGGREGATE FUNCTION
↓
HAVING
↓
SELECT
↓
ORDER BY

Having EX: Select Course, AVG(marks)

From students

Group by Course

Having AVG(marks) > 80;

It executes in: from students table,

Group by Course,

then calculate AVG(marks), then check condition "having",
at last display results.

Ex: Select Course, Count(*)

from students

group by course

having count(*) >= 3

SQL Syntax has fixed order

we must write in select

From

Group by

having

order by

Execution order: From

where

Group by

Aggregate function

having

select

ALIASES (AS)

When we execute `select count(*)`
from students;

the output will be

No column name
30

`select avg(marks)`
from students

~~no column name~~

81

It looks some unprofessional method so to overcome this we rename the output column

`select avg(marks) AS AverageMarks`
from students;

output:

Average marks
81

Definition:

An alias is a temporary name given to column or table in result set.

- It does not change the original/actual column in table
- It changes only in output.

`select count(*) AS TotalStudents`
from students;

① `select studentName as name,`
`marks as score`
from students

} → in this the original table has studentName & marks but only in output it shows name and score.

Alias with Group by

`select course,`
`count(*) AS TotalStudents`
from students
group by course;

Alias with all aggregate function we can use

`count()` as
`min()` as
`max()` as

⇒ we can skip AS, but it will be easier to read,

more professional,

used widely in companies if we ~~use~~ use it.

Between

If manager asks show all employees whose salary is between 40000 & 60000.

1st method

Select * from Employees
where salary >= 40000
AND salary <= 60000;

2nd method (easy)

Select *
from employees
where salary BETWEEN 40000 AND 60000;

Both gives same result

Definition: Between is used to filter values within specific range, and it includes both starting and ending values.

→ it includes starting & ending values that means if an employee earns exactly 40000 & 60000 they are included
it is same like $>= 40000$ AND $<= 60000$.

Imp Rules: Between:

Between 70 AND 90

means includes both 70 and 90 also

NOT BETWEEN

It gives out of range

ex: NOT Between 70 AND 90
gives out of 70 and 90.

⇒ Between can also used with

IN: If manager asks show students who are enrolled in either SQL or Python.

for this we have

1st method

Select * from Employees
where course = 'SQL' or
course = 'python'.

is between
40000 & 60000.

But if there are 10 courses, we have to write ~~where~~

where course = 'SQL' or
course = 'python' or
course = 'Excel' or
course = 'Java' or
course = 'C'.

So to overcome this we have,
"IN".

Select * from Students

where course IN ('SQL', 'Python'); (In english: show students whose course is IN this list.)

Definition: In is used to check whether a value matches any value in specified list.

when we write

~~Select * from~~ where course IN ('SQL', 'Python')

internally it thinks

where course = 'SQL' or course = 'python'

⇒ So "IN" is more cleaner & more readable way to write multiple "OR" conditions.

NOT IN: Shows students not enrolled in SQL or python

where course NOT IN ('SQL', 'Python') ↗

Suppose if we want the employees/students whose names start with 'H' or ~~any~~ any letter.

Firstly we filter the rows where name starts with 'H'.

⇒ So for this problem we have special operator for pattern matching.

LIKE

% ⇒ Zero or more characters can come here

↳ Anything after this

Ex: Select * from employees

where employeeName like ~~##~~ 'H%';

↳ In english: show empName starts with 'H'.

ex: ② select * from employees
where empname like 'A%'

} → show names starts with A

ex: ③ where empname like '%H' } J of. comes before letter
(This means ends with h.)

O/p: Rakesh
mahesh
suresh

④ where empname like '%ar'; }

Contains 'ar' anywhere

O/p: Harish
Harsha
Anjun

→ SQL server uses "like" with character data

→ If phone number is stored as varchar, it works perfectly.

ex: 987654 like 98%

Like is designed for text pattern matching, not numeric comparisons

→ Think "like" as text-searching tool

Char	Like ✓
Varchar	Like ✓
Text	Like ✓
Int	Like x
Decimal	Like x

_(underscore)

Cost means only one character

unlike % means 0 or more characters

~~NULL~~ IS NULL & IS NOT NULL

Ex: Name phone
Harish 987654
Rahul NULL
Sai 912345

NULL means the value is missing or unknown.

Starts with 9

In real-life ex: Imagine a form

Name	Phone
Hansh	987654
Rahul	—

Rahul didn't write his phone number

Did he write NULL? X

He left it empty

So SQL stores it has NULL, that means unknown or Missing

SQL treats "NULL" as "Unknown"

So if we write where phone = NULL

it does not return any rows

So for this we use special operator IS NULL

where phone IS NULL

→ It checks whether phone value is missing

IS NULL is not comparing,

it just checking its state.

If we write

Select * From
Students
where phone = NULL;

SQL Server

987654 = unknown?

False

Unknown - Unknown?

I don't know

NOT True

d/p: no rows returns

Now 'is null'

Select * from
Students

where phone is null;

In SQL NULL means something unknown or missing

ex. If I ask how much age is Rahul

You say I don't know

In SQL that is NULL

You didn't say 0 years old

or
NULL years old

I don't know

Suppose person's A age is unknown

B age is "

If I ask are they have equal age
then that could be may or may

So we can say unknown - unknown not
no, that is false

SQL uses Three valued logic

TRUE
FALSE
UNKNOWN

Most programming lang uses TRUE
FALSE

5=5, True

5=7, false

NULL=NULL, Unknown

Why doesnot "where" return "Unknown"

→ where returns rows where condition is true.

If result is false, no rows

is unknown, no rows

Then why is NULL?

Instead of asking: Is phone equal to NULL?

It asks: Is phone missing?

Hari

phone: 988052

Is phone missing

NO

Rahul

phone=NULL

Is phone missing

Yes

→ It returns as result (Rahul)

UPDATE

Syntax: UPDATE TABLE-NAME

SET column-name = new value

where condition;

Ex: UPDATE students

set marks = 95

where student-ID = 101;

→ first find row
where Id = 101,

→ change 85 to 95

→ Done //

→ Only one row changed

If there is
no where condition

then it updates every row
of that column ⇒ UPDATE students

set marks = 95;

name

Hari 95

Rahul 95

Rule: Always check your "where" clause before running an "update"

- 1, First run a select query with some "where" condⁿ
- 2, verify rows
- 3, then update

UPDATE (Advanced)

Part-1 updating multiple columns

Ex Table:

ID	Name	Course	Marks
1	Hanish	SQL	85
2	Rahul	Python	72
3	Sai	SQL	90

change Hanish: Course $\rightarrow$ Python
marks = 95

So, should we write two update queries for this? no.

we update it like $\downarrow$

UPDATE students SET Course = 'Python', marks = 95
where studentID = 1;

Part-2, Increasing value

Suppose if you want to increase marks by 5

then, Update students set marks = 5; } No, it updates everyone's to 5.

$\Rightarrow$ Update students

Set marks = marks + 5;

updated marks

Hanish	90
Rahul	77
Sai	95

Part-3, Increase only SQL Students

Update students

Set marks = marks + 5

where course = 'SQL';

Ex: Salary Hike for 'IT' employees

Update Employee

Set Salary = Salary + 5000;

where dept = 'IT';

Golden Rule: for update & delete
* whenever possible, use primary key in where condition

DELETE

↳ delete rows from table

Suppose example we have student table,
if rahul leaves the course,
we should delete/his record
remove

Syntax: Delete from table name
where condition;

Ex: Delete from students
where student-ID = 2;

if we write → delete from students;

then the server thinks:

"delete which rows?"

then: All rows

Then the table structure remains, but every record is deleted

* So for delete,

first run a select query with same where clause

Differences:

Command	Deletes data	Structure of table
Delete	Yes	NO (DML)
Truncate	Yes	NO (DDL)
Drop	Yes	Yes (DDL)

where conditions

^{DOL} Delete command delete selected rows & table still exists.

Truncate: Truncate table table_name;

↳ Deletes all rows, but keeps columns, constraints & datatypes.

Table structure is exists

you cannot use where.

DROP: Drop table table_name, Drop database database_name;

↳ Delete entire table, rows, columns, constraints.

ALTER TABLE

Alter table is used to modify the structure of an existing table without deleting table or its data.

→ It changes the structure not data.

→ Suppose if we have employees table where we need to add phone numbers.

A) SO, should we delete the entire table & create it again.

no we have Alter table.

What can alter table do?

→ It can add new column

↳ remove column

modify column

Add constraints

remove constraints

Add Column:

Table	EMP ID	EMP NAME	Salary
	101	Harish	50K
	102	Rahul	45K

if manager says add column "phone".

Syntax:

Alter table Employees

Add phone number Varchar(15);

case server & dropping

add new column

↓
don't touch existing rows

↓
structure updated.

After table struct:

emp_id	name	salary	phone
			NULL
			NULL

Phone numbers is null, because the column is new & SQL doesn't know anyone phone numbers.

Add multiple columns:

Alter table table-name

add column-name1 datatype,

column-name2 datatype;

⇒ All new columns initially contain NULL

⇒ Alter table changes structure, ~~do~~ not delete rows, ^{not} update salary.

Alter table Part-2

Drop column & Alter column

→ If your manager says there is no need of ~~year~~ ^{age} column

Should we create a new table X
for delete column?
delete the table X

Drop Column: is used to remove one or more columns from an existing table.

It removes only column, does not delete table.

Syntax: Alter table Employees

Drop column Age;

Alter table table-name

drop column column-name;

SQL server 2
Employee table

↓
find column "Age"

↓
Remove entire column

Alter column

Syntax: Alter table table name

alter column ^{existing} column name new datatype

→ Suppose if we want to change the data type of a column we use this.

like Salary INT → Salary decimal(10,2);

JOINS:

Suppose you are working at e-commerce,

if your manager asks to show custname along with product they ordered where you have cust table and order table

Output must be

custname	Product
Hanish	Laptop
Rahul	Mouse
Hanish	Keyboard

→ Data in relational databases is stored across multiple related tables to avoid redundancy. When we need information that exists in ~~one~~ more tables, we use JOIN to combine those tables based on common column.

→ If manager wants information from both tables we use/solve it:

SQL solves: Customer table

↓
order table

↓

is there any column common?

↓

Yes

↓

Cust-ID

↓

match cust-ID

↓

Combine Information

↓

Show result

we are not permanently combining the table, we are only creating temporary result, for that query.

SQL server do?

ID	Cust. Name	order table ID
1	Harish	101
2	Rahul	102
		103

Cust. ID	Product
1	Laptop
1	mouse
2	Keyboard

SQL server starts with first customer

CustID = 1

then look into orders table

find CustID = 1?

Yes

Product

Laptop

mouse

C Name	Product
Harish	Laptop
Harish	mouse

Same it repeats for entire table

the final output is

C Name	Product
Harish	Laptop
Harish	mouse
Rahul	Keyboard

Ex: Select customerName, Product

FROM Customers

Inner JOIN orders

ON Customers.customerID = orders.customerID;

⇒ A Join is possible when SQL server knows how two tables are related

That relationship is created using

→ primary Key (in one table)

→ Foreign Key (in another table)

Query:

→ select customername, Product

means: I want to display customername & product in output

→ From customers

means: start with customers table

SQL server opens customer table

→ Inner Join orders

means: now also bring orders table.

→ It says need orders table

→ on customers. CustomerID = orders. CustomerID

means: match the rows where custID is same in both tables.

This is where actual joining happens.

no. of rows in an inner join is not based on number of customers.

→ It is based on number of matching records

Rules of Join

→ Take first row from left table

→ look for same ID in right table

→ If found → create rows

→ If not found : Inner join → skip it

left join → create a row with null.

Right Join : If the manager wants ~~all~~ ^{all} orders, without losing any.

Tables:

customers		orders		
ID	Name	ID	cust ID	Product
1	Harish	101	1	Laptop
2	Rahul	102	1	mouse
		103	5	printer

which table contains all orders? order table

→ So server thinks, keep every row from orders table even if there is no matching customer.

Query: Order 101
cust ID = 5
Search customer
found Harish

Name
Harish

Product
laptop

"102

Harish

Mouse

103

cust ID = 5

Not found

But manager said

Keep all orders

So server cannot remove this row

So, it creates

Name	Product
Null	Printer

Right JOIN

Name is Null, because
the SQL server found
order but not customer
matching.

final o/p:

Name	Product
Harish	Laptop
Harish	Mouse
NULL	Printer

Inner Join

Harish	Laptop
Harish	Mouse

Left Join

Name	Product
Harish	Laptop
Harish	Mouse
Rahul	NULL

Trick: From customers

Right Join orders

Read { Customers → left table
it { Orders → Right table

Right Join always keeps every row from Right Table.

Similarly:

Left Join → keep every row from left table

Inner Join → keep only rows that match in both tables

JOIN → Bring another table.

ON → Tell SQL server how to match tables

AS → Give temporary short name.

Trick: which table rows do you need?

if you want customers → put customer table on left & use left join

if you want orders → put order table on left & use left join

FULL OUTER JOIN:

cust ID	Name	orders ID	cust ID	product
1	Harish	101	1	Laptop
2	Rahul	102	5	printer
3	Sai			

If manager says I don't want to lose anything. show me every customer & order, even if they don't match also

so the SQL server checks all the rows & final result is

JOIN

Product
Laptop
Mouse
Printer

Name	product
Harish	Laptop
Rahul	Null
Sai	Null
Null	printer

Comparison:

Join type	Keeps
INNER JOIN	Only matching rows
LEFT JOIN	All rows from left table + matching rows from right
RIGHT JOIN	All rows from right table + matching rows from left
FULL OUTER JOIN	All rows from both tables, unmatched becomes NULL.

"ON" clause tells SQL server how the two tables are related by specifying the matching columns.

"AS" is used to make queries faster to write, especially when joining many tables.

Text functions

left(): gives ~~string~~ characters from left of string

Syntax: `left(string, 3)`

returns 3 characters from string from left side

Ex: `left('harish', 3)`

o/p: Har

Right(): returns characters from right

Syntax: `Right(string, number of characters)`

Ex: `Right('harish', 3);`

o/p: ish

Len(): returns the number of characters in a string.

Syntax: `select len(string);`

Ex: `select len('harish');`

o/p: 6

Ex: `select Name, Len(Name) as Total characters`
from Employee

Name	Total characters
harish	6
Rahul	5

Length ignores trailing spaces (that are last)
but counts spaces in between.

UPPER() Converts text into Upper case

Ex: `select upper('harish')` o/p HARTISH

Ex: `select upper(Name) from Employee;`

Lower(): Converts text into lowercase

`select lower('HARTISH');` o/p harish

Substring():

extracts text from any position

Syntax: Substring(string, start position, length)

ex: Select Substring('Hyderabad', 1, 5);

H Y D E R A B A D
1 2 3 4 5 6 7 8

O/p Hyder

in this it starts from 1st position & gives 5 characters & then return them.

LTRIM(): removes spaces from left

RTRIM(): removes spaces from right

TRIM(): removes spaces from both sides

Replace(): replaces text, replaces all matching text

Syntax: Replace(string, old, new)

ex: Select REPLACE('SQL Server', 'Server', 'Database')

O/p: SQL Database

CONCAT: Joins multiple strings

Select Concat('Harish', ' ', 'Gunderani'); O/p: Harish Gunderani

Charindex(): find the position of a character or word

Syntax: CHARINDEX(search_text, string)

ex: CHARINDEX('@', 'harishgunderani@gmail', 1)

O/p: 7

position from where to start the search

Stuff():

It works using positions not text

Syntax: Stuff(string, start position, no. of char (length), new_text)

Go to this position

↓

Delete this many characters

↓

Insert new text

Select Stuff (Harish, 4, 0)
Positions H a r i s h
1 2 3 4 5 6

Go to 4
Delete 3 characters I s H
insert XXX

O/p: HarXXX

Ex: Select Stuff('abcdef', 3, 2, 'XYZ')
a b c d e f
1 2 3 4 5 6

a cd delete

insert XYZ

O/p: aXYZed

Replicate(): repeat a character or text

Ex: select Replicate('*', 5);

O/p: *****

Ex: select Replicate('AB', 3);

O/p: ABABAB\$

Whenever we know the ending position

*
**
$$\text{Length} = \text{End} - \text{start} + 1$$

Suppose

Ex: AB 1 2 3 C D
1 2 3 4 5 6 7

we want 123

So we know where to start 4 end

Start = 3, end = 5

Length = $5 - 3 + 1$

= 2 + 1

= 3

Golden Rule

There are only 2 situations in SQL string problems

Situation 1) You know actual ending character

Ex: AB123CD

o/p: 123

we know last digit is at 5th position

Formula: $\text{length} = \text{end} - \text{start} + 1$

$$\text{Ex: } = 5 - 3 + 1$$

$$= 3$$

Situation 2: your start & end markers are separators.

Ex: ^{1 2 3 4 5 6 7 8 9 10 11 12}HYD-IT-2026-001

The hyphens are not part of result

Formula: $\text{length} = \text{end separator} - \text{start separator} - 1$

Ex: If we want "2026" from ex

$$\text{length} = 12 - 7 - 1$$

$$\text{length} = 4$$

$$= 2026$$

What is SubQuery?

A Query inside another Query

Syntax: Select ...

From --

where $\text{column} = (\text{select ...})$

it is like

main Query

→ Before answering, it asks another Query

→ Gets Value

main Query uses that value

ex. Imagine your mom says:
 Bring the student who got highest marks
 Can we know ~~you~~ who directly is? no
 first find highest marks
 then find student whose marks = highest marks

ExTable

ID	NAME	SALARY	DEPT
1	Hemish	25000	IT
2	Ravi	40000	HR
3	Rahul	35000	IT
4	Anil	45000	SALES
5	Kiran	30000	HR

18) Display highest salary

select max(salary) from employee; o/p 45000

20) Display emp who earns highest salary

Do we know highest salary?

no

first find 45000 then compare

Step 1: select max(salary)

from employee;

o/p: 45000

Step 2: select * from employee

where salary = 45000; o/p: Anil

now combine them

select * from employee

where salary =

(select max(salary)

from employee

);

→ SQL first runs inner query & then outer query

Subquery works like this

Inner Query

↓
output

↓
Replace output

↓
Read outer Query

Q) display employee working in ravi's dept

select dept from employee

where emp-name = 'ravi'

o/p HR

select emp-name from employee

where dept = (select dept from employee

where emp-name = 'Ravi')

o/p : Ravi,
Kran

Q) display employees earning more than Ravi

First Select salary from employee

And Inner: where empname = 'Ravi'

o/p = 40000

Outer:

Select empname, from employee

where ~~emp~~ salary > 40000;

Combined:

Select * from employee

where ~~empname~~ salary > (select salary

from employee

where emp-name = 'Ravi');

Types of subqueries:

1) Single row subquery

Returns one value

Operators used like >, <, >=, <=, =

2) Multiple Row SubQuery

Returns many values

Ex: Select Dept from employee

o/p: IT

HR

Sales

HR

IT

Now we cannot use '='

Instead we use IN, ANY, ALL, EXISTS

4-step formula: for every subQuery Question

Step 1: Ask what value do I need first

2: write inner Query to get the value

3: Run it mentally & note the result

4: write outer Query using that result

Level 2: Single row vs multiple rows subQueries

Suppose I ask: Show emp whose salary is equal to highest salary
what does subQuery return

Select max(emp-sal)

from emp

o/p: 50000

Output only one value

So we can write

where emp-sal = (select max(emp-sal) from emp)

now another Question

Show employees working in IT or HR dept

Suppose
if we write

Select * from emp

where emp-dept =

(select emp-dept from emp

where emp-dept = 'IT' or emp-dept = 'HR'

)

lets see what subquery returns (Conces Query)

```
select emp-dept  
from emp
```

```
where emp-dept = 'IT' or EMP_Dept = 'HR'
```

O/p: IT

HR

IT

HR

How many values, more than one

Can SQL do this? no

```
where emp-dept = ('IT', 'HR', 'IT', 'HR')
```

NO

It gives an error

for this problem we have solution $\rightarrow$ IN

Instead of $=$, we use IN

```
select * from emp
```

```
where emp-dept IN
```

```
(select emp-dept from emp
```

```
where emp-dept = 'IT' or emp-dept = 'HR')
```

for list of values we use "IN"

Remember

One Value

multiple values

Use

$=, >, <, >=, <=, <>$

IN, NOT IN, ANY, ALL

Q) emp salary $> 30,000$

```
select emp-dept from emp
```

```
where emp-sal  $> 30000$ 
```

} $\rightarrow$ returns dept > 30000 having salary

```
select * from emp
```

```
where emp-dept IN (select emp-dept from emp
```

```
where emp-sal  $> 30000$ )
```

O/p: we get emp depts & emp who are having $> 30,000$ salary

2) Select emp-sal from emp
where emp-name = 'Ravi'

} returns only one value

select * from emp
where emp-sal = (select emp-sal from emp
where emp-name = 'Ravi')

3) EMPNAME EMP-DEPT

Ravi	HR
Harish	IT
Rahul	IT
Kiran	HR
Anil	Sales

Q) Show emp working in dept where Harish & Ravi work

Select emp-dept from emp
where emp-name = 'harish' or emp-name = 'Ravi' }

gives 2 values of O/p.
So use "IN"

select * from emp
where emp-dept IN (select emp-dept from emp
where emp-name = 'harish' or emp-name = 'Ravi')

for multiple subquery we use "IN"

⇒ see for example if emp working in dept Harish & Ravi

Select emp-dept
from emp
where emp-name in ('Harish', 'Ravi')

O/p: IT

HR

O/p is two rows, can we use '=' no

so we use "IN"

"IN" acts as the value present inside the list?

Ex: show emp whose salary = Ravi sal or Harish salary

Inner Query { Select emp-sal
from emp
where emp-name IN ('Harish', 'Ravi')

Query: select * from emp
 where emp.sal ~~IN~~ (select emp.sal from emp
 where emp.name IN ('Ravi', 'Harish'));
 "NOT IN" works opposite to "IN"

Ex: Show emp not working in empname=Harish & Ravi
 select * from emp
 where emp.dept NOT IN (select emp.dept from emp
 where emp.name IN ('Harish', 'Ravi'))

Exists:

Exists care about whether a matching row exists
 exists does not look at returned value, it only ask for at least one matching row.

<u>IN</u>	VS	<u>EXISTS</u>
Is the values present in the list		Does a matching row exists

Correlated SubQuery

Ex: Imagine you are teacher
 you have 6 students

Student	Class	marks
Ravi	A	80
Harish	A	90
Anil	B	70
Rahul	B	60
Kiran	C	95
Ajay	C	85

If anyone ask find students whose marks are greater than their class average

not we do them manually

→ Harish

Class A

In class A we have two students

their Avg = 85, now compare 80 > 85

NO, so next

Ravi,
Class - A
In class A we have two students
their avg 85, compare 90 > 85
Yes

like this ~~repeat~~ repeat for all,
we get students having ~~avg~~ marks greater than their class avg

This is exactly SQL does:

Take one emp

↓
find his dept

↓
Calculate avg of that dept

↓
compare

↓
go to next emp

↓
Repeat

Window Functions

why did microsoft create window functions when we already have
"group by"

→ See to display the avg (salary) of each department
we use group by where it reduces no. of rows
and group by into 2/3 rows/some rows

ex:

dept	avg
IT	60000
HR	35000

→ Per Suppose if i want every employee along with their
~~avg~~ average salary of their department.

Op must be like this:

Name	dept	salary	dept. AVG
Harish	IT	50000	60000
Ravi	IT	60000	60000
Anil	IT	70000	60000

Rohit	HR	30000	35000
Kiran	HR	40000	35000

See here we have every emp-name & their dept-avg beside each employee
 ⇒ group by cannot do this,

it only returns one row per dept, not one row per employee.

⇒ GROUP BY collapses rows

ex: 5 emp rows



Group By → 2 dept rows

⇒ The biggest limitation of group by is reduces no. of rows

⇒ But if I don't want to lose my rows,

⇒ so to overcome this we have window functions.

Simple Definition

A window function performs calculations across a group of rows without reducing the no. of rows returned.

In other words:

group by → groups rows & returns fewer rows.

window function → keeps all rows & add calculated values.

Window functions think differently

⇒ Instead of compressing the rows, it keeps every row.

⇒ Suppose manager asks to show every employee along with their dept avg.

⇒ group by can't do it,

it does avg for every group & removes rows keeps only some rows.

⇒ By using windows we can do it without removing rows.

like:

name	Sal	avg
Hanish	5000	6000
Ravi	6000	6000

Ex: {

This is why OVER() was created

Syntax: select emp-name, emp-dept, emp-sal,
AVG (emp-sal) over (Partition by emp-dept) as dept-avg
FROM EMP

Let's read

AVG (EMP-SAL)

It means calculate Avg (Emp-Salary)

OVER (PARTITION BY EMP-DEPT) $\Rightarrow$ Calculate separately for each department

$\hookrightarrow$ means don't calculate one average for whole table,
Calculates separate avg for each department.

What does PARTITION mean?

Partition means split into groups

Ex: Harish IT

Ravi IT

Anil HR

Rahul HR

Kiran Sales

Kiran Sales

Partition BY EMP-dept splits table into

Partition 1

Harish

Ravi

Partition 2

Anil

Rahul

Partition 3

Kiran

Ajay

SQL does not create new tables

$\rightarrow$ It just mentally treats them as separate groups.

main difference

Group by rows $\rightarrow$ 3 rows

Partition by rows $\rightarrow$ 6 rows

ROW_NUMBER()

Emp ID	NAME	SALARY
101	Hansh	50000
102	Ravi	70000
103	Rahul	60000
104	Anil	40000

$\Rightarrow$ now if your manager asks display emp from highest to lowest salary

Select * from emp

order by emp-sal desc;

Ravi	70000
Rahul	60000
Hansh	50000
Anil	40000

$\Rightarrow$ now the manager asks to give them a position
like it must be

Position	Empname	Salary
1	Ravi	70000
2	Anil	60000
3	Hansh	50000
4	Rahul	40000

Here, the no.
are not stored in table
SQL generated them

This is exactly what

Row-number() does:

Ex. syntax: select empname, emp-sal,

Row-number() over (order by emp-sal desc) as position

From emp;

$\Rightarrow$ Break down (read it)

① Row-number()

$\hookrightarrow$ gives row numbers

② over (...)

means how to assign those row numbers

③ order by emp-sal desc

means, first arrange emp (high to low salary), then assign row numbers

row-number always gives unique numbers
↳ gives unique sequential no.

for example: select emp-sal, emp-name,
~~row-number()~~ row-number() over (order by emp-sal) from emp

→ It gives row numbers for no. of rows without repeating,
if they have same salary also

RANK():

Ex we have students, marks table

Now we want ranks for these
students,

So if we use row-number() it
gives unique sequential number

But, Harish and Ravi got high marks

there rank should be '1'

student	marks	Row	Rn
Harish	95	1	1
Ravi	95	2	1
Anil	80	3	3
Rahul	85	4	4

⇒ so to overcome this, we have Rank()

Rank() says: If two people have same value, they get same rank

⇒ Here the next upcoming ranks is skipped & next value is assigned

Rank() ⇒ Duplicates are allowed, Skipped ranks are possible

~~But microsoft engineers ha~~

So to overcome this Rank() skipped ranks, we have
another function Dense-rank()

Ex:

Student	marks	Row ^{num}	Rank()	Dense-Rank()
Harish	95	1	1	1
Ravi	95	2	1	1
Anil	80	3	3	2
Rahul	85	4	4	3
Shiva	85	5	4	3
Ram	88	6	6	4

Row-number() unique numbers
does not care about ties

RANK() Same value = same rank
skips the next rank

Dense-rank() : Same value = same rank
NO Skipped rank

SQL Execution order

FROM

↓

WHERE

↓

GROUP BY

↓

HAVING

↓

WINDOW

FUNCTIONS

↓

SELECT

↓

ORDER BY

Lag()

Suppose you have monthly sales

month	Sales
Jan	100
Feb	120
Mar	150
Apr	130

If manager asks: for every month, show this month sales & previous month sales

expected old month	Sales	Previous Month Sales
Jan	100	NULL
Feb	120	100
Mar	150	120
Apr	130	150

The word `lag()` means "go backward"
`LAG()` means, it works like get previous row value

Syntax: `LAG (Emp-Sal) over (order by month)`

Read it in english

① `lag(emp-sal)`

means give me previous salary

Then

② `over (order by month)`

means while moving through the rows in month order.

SQL executes the table month like

SQL Reads Jan

Current Row Jan

Previous Row?

None

Return NULL

SQL Read Feb

Previous Row?

Jan

Return 100

SQL `select month, sales,`

`lag(sales) over (order by month) as previous sales`

`from sales;`

⇒ The previous row depends entirely on the "order by" inside the window function.

Lead()

means look forward

month	Sales
Jan	100
Feb	120
Mar	150
Apr	130

Query: Select month, sales,

lead(sales) over (order by month)
as next-month-sales
from sales;

Step 1 order by month

Current row = Jan

next row

Feb
sales 120

Step 2 Current row = Feb

next row = Mar

Return 150

Step 3 Current row = Apr

next row = no next row

return null

Final output:

month	Sales
Jan	120
Feb	150
Mar	130
Apr	NULL

Comparison:

function

lag() → Previous row

lead() → next row

RUNNING TOTAL (SUM() OVER())

lets understand the business problem

Imagine you own a shop

Sales/day	Sales
1	100
2	200
3	300
4	400
5	500

your manager asks: how much you have earned till today

If someone asks, earn in total

Ans: $100 + 200 + 300 + 400 + 500$

= 1500

one value

We already know `Select sum(sales)`

from sales;

o/p: 1500
only one row

But manager asked till today
so far till day 1

Ans: 100

then till day 2

Ans: 200

no, because he asked

till day 2 that means $100 + 200 = 300$

then day 3 that means $100 + 200 + 300 = 600$

day 4 " " $100 + 200 + 300 + 400 = 1000$

day 5 that means $100 + 200 + 300 + 400 + 500 = 1500$

final output

Day	Sales	Running total
1	100	100
2	200	300
3	300	600
4	400	1000
5	500	1500

} This is not sum()
this is running sum

⇒ Here every row is remembering
all previous rows

Normal sum()

100
200
300
400
500

1500

One sum

Running sum

100
↓
300
↓
600
↓
1000
↓
1500

} many answers

If I want till day 3, SQL adds $100 + 200 + 300 = 600$

till day 5, SQL adds $100 + 200 + 300 + 400 + 500 = 1500$

Window - thinking

This is where the word
window becomes very important

Day 1
Window
Sum: 100

Day 2
Window grows
[100, 200]
Sum: 300

Day 3
Window [100, 200, 300]
Sum: 600

→ The window keeps
expanding as SQL
moves down the
row

What does OVER() means

OVER (Partition by dept) $\Rightarrow$ window = each department

OVER (Order by salary) $\Rightarrow$ window grows row by row according to order.

when we have:

OVER()

there is no partition

there is no order by

So what is the window?

the entire table

Imagine SQL saying:

Current Row = Day 1

Window

everything

Day 1

Day 2

Day 3

Apply:

Sum(Sales)

Result: $10 + 20 + 30 + 40 = 100$

Day 2

Window

everything

Day 1

Day 2

Day 3

Sum: 100

Final output:

Day

Sales

Sum(Sales) over()

1

10

100

2

20

100

3

30

100

4

40

100

Easiest way to remember:

Case 1: Sum(Sales) over()

Window: [whole table]

O/P: 100

100

100

100

100

Case 2:

over(order by day)

Window grows

[day 1]

10

[day 1 day 2]

10, 20 = 30

[day 1 day 2 day 3]

10, 20, 30 = 60

[day 1 day 2 day 3 day 4] 10, 20, 30, 40 = 100

O/P

day

1

2

3

4

Running

Subtotal

10

30

60

100

Sum() over() is not always a running total
It can do diff things depending on what's inside over()

Query:

Sum(Sales) over() total of table, repeated on every row

Sum(Sales) over (order by Day) Running total (Cumulative sum)

Sum(Sales) " (Partition by year) total within each year, repeated for rows in that year

Sum() over() => for every row calculate sum over entire table

Window functions execution order

Partition by



order by



Apply window function

EX: Rownumber() over (Partition by dept order by salary Desc)

execution order:

partition



Sort



Assign row numbers

AVG() is also same as sum()

Count() also works same as sum()

	over()	over (Partition by)	over (order by)
Sum()	total of whole table total given to each row	Total per partition + each row same value	Running total
AVG()	Average of whole table	Avg per partition	Running avg
Count()	total rows in table	rows per partition	Running count

why window function are invented?

Suppose you are working in amazon and you are manager and
to show every emp details like name, salary.

then i use: `select empname, salary from emp;`

But when he ask for max salary

then i use: `select max(salary) from emp`

But when he ask show every employee & company max salary
with all employees

then what to do.

in normal first we calculate the max salary and then

add to empname so it will be like this

Amanish	3500	50k
Ravi	3000	50k
Rahul	2800	50k

⇒ so when we wanted max salary and
display all employees and then we

use `select empname, max(salary) from emp`

we get error, because there is no group by and it
reduces rows

because SQL is confused

now SQL have `max(salary)` for which employee should i print it

like Amanish 50000

or Ravi 50000

or Rahul 50000

So That it gives error

So SQL have 2 choices ① group by
merge rows

but employees are reduced

② Keep every employee

& still calculate max

This is why window functions are invented.

⇒ A window function lets SQL perform calculations without removing the original rows

Let's compare

① GROUP BY

Show every emp & company avg salary

Select emp name,
Salary, Avg(salary)
from emp;

⇒ it gives error

→ SQL says you are asking to calculate one avg salary then why you are asking to print every employee

So we use group by 'it reduces rows

Select dept, Avg(salary) from emp
group by dept

dept	Avg Salary
IT	60000
HR	25000

Here rows are disappeared
no Harish, Anil, Ravi

② Window function

⇒ Show every emp &
Company avg salary

Window function says: I can do both
& I won't remove any employee

q/p:

emp name	Salary	Avg
Harish	50000	50000
Ravi	40000	50000
Anil	60000	50000
Rohit	40000	50000
Sita	30000	50000

So why window function invented?

It was invented because

I want to calculate something

but I don't want to lose my original rows.

OVERC) $\Rightarrow$ tell SQL "these are the rows you should look at before calculating the function"

Partition by does not calculate anything it just divide file rows into minitables]

Current row
find partition
Arrange rows in order
then apply function

which rows belongs to me?
 $\Rightarrow$ Partition by

In what order should those rows be seen
 $\Rightarrow$ order by

what should I do?

AVG, sum, Row no. lead, ...

Window function Algorithm

- ① Current row
 $\downarrow$
- ② find my partition
 $\downarrow$
- ③ Take all rows in my partition
 $\downarrow$
- ④ If orderby exists
arrange them
 $\downarrow$
- ⑤ Apply function
 $\downarrow$
- ⑥ return value